Unit 1 – Using Objects and Methods
Lesson 5: Creating and Storing Objects



Adding Classes

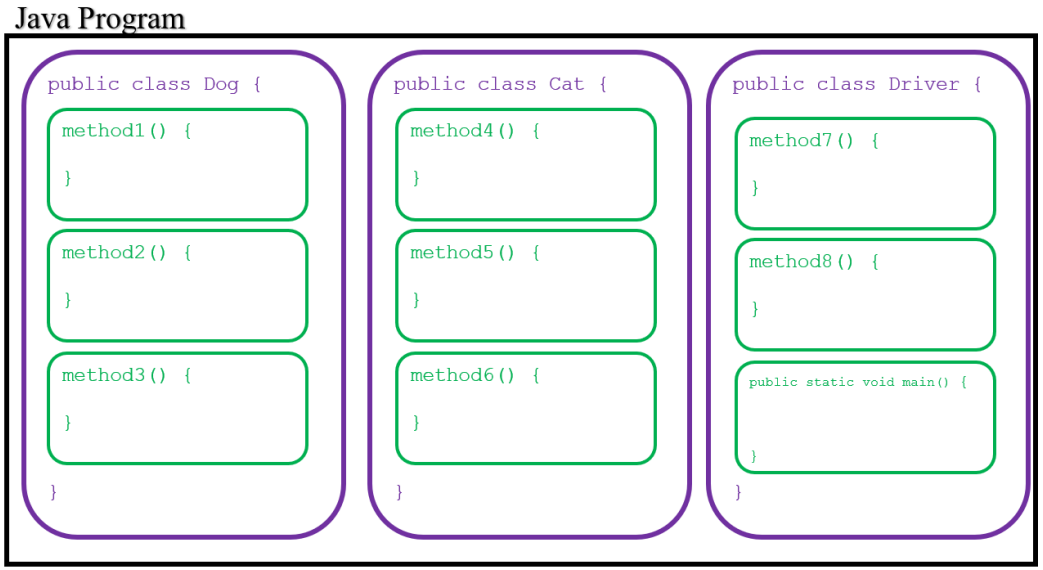
Up to this point, everything we have coded has been in a single `main` method inside of a single class:

```
public class Example
{
	public static void main(String[] args)
	{
		System.out.println("Hello World!");
	}
}
```

- A program can have many _____ in it.
- In a Java program, the _____ method is the only method that will run.
- Other classes and methods will _____ unless they are called on from the main method.

OOP - Object-Oriented Programming

- A style of programming where you:
  -
  -
  -
- This paradigm puts _____ at the center of your program instead of logic.
- Everything you do in OOP is centered around data and how data interacts with each other.
- Data takes the form of _____



Classes as Objects

- A class is a template for the _____
- A class defines what attributes the object will have as well as what its behavior will be
- For example, let's say we have a Dog object. A dog has attributes and behaviors that are common to all dogs. We define these in a class using instance variables and methods

Attributes (_____):	Behavior (_____):
• `String name` • `int age` • `boolean goodDog`	• `speak` • `haveBirthday`

Example: Dog Class

- A `Dog` class is a blueprint for _____
- Any `Dog` we create will have these _____ and these _____
- The state might be different for each dog, but they will all have these attributes
- Each dog might behave differently, but each dog has these behaviors

Below is the general template for an object-creating class, with comments where the code will eventually go

```
public class ObjectName
{
     //instance variables

     //constructors

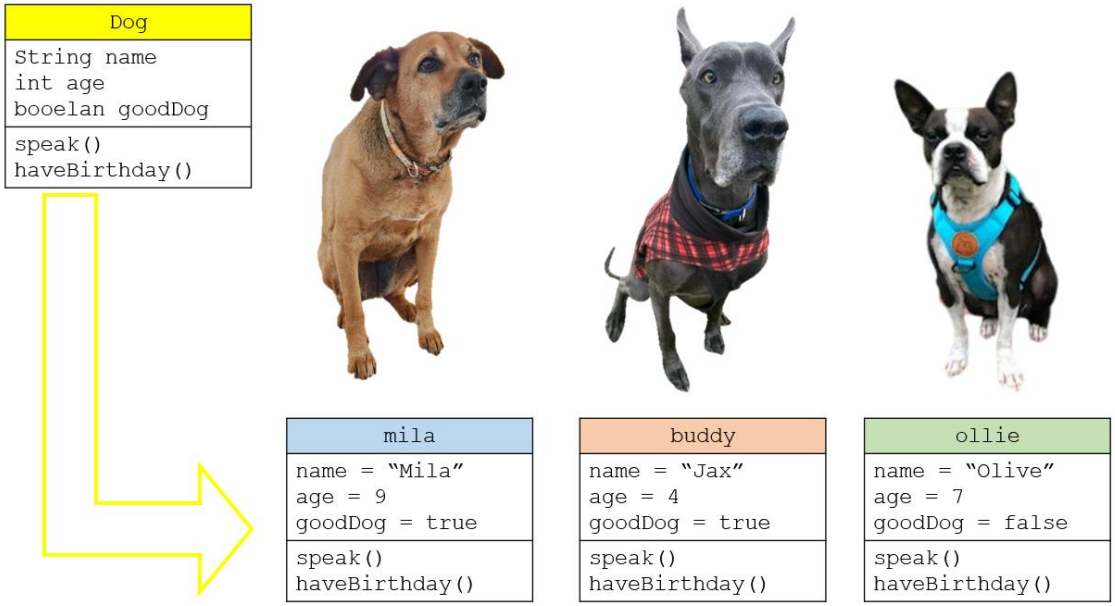
     //accessors and mutators

     //toString and equals

     //behavior methods
}
```

Creating Instance Variables

- _____ define the data that your object will be described with.
- You want instance variables to be _____. This is a pillar of object-oriented programming.
- _____ is protecting data from being manipulated by methods and classes that you don't want to change it.
- By setting the instance variables to "private" you are saying that no other class beside the object creating class can manipulate it.
- Instance variables are considered _____, meaning you can use them freely throughout your class.



Creating a Constructor

_____

- This constructor will _____ (create) your object by providing space in memory for the instance variables and setting them equal to default values
- You want this to be _____ because other classes will need to call upon it to create objects
- There is no _____ and no _____
- The constructor must have the _____ as your class
- There is _____ default constructor in a class

_____

- This constructor will serve the same purpose as the default, except you will pass it _____ to set the instance variables to
- You will use this constructor when you have data you want to create your object with
- The parameter constructor should be _____ with no _____
- Inside the parentheses of the constructor are _____, which will be assigned to the values of the instance variables
- There can be none, one, or many parameter constructors in your class

Dog Class with Instance Variables and Constructors

Class Header	
Instance Variables • Declared _____ of your class • Do not assign any values to these, that is the job of the constructors • You can have as many instance variables as you want	
Default Constructor • Same name as the class • A default constructor has no _____ • Inside the constructor, the instance variable are assigned to default values chosen by the programmer. • You would use this constructor when you want to reserve space in memory for a dog object, but don't know anything about the dog yet	
Parameter Constructor • There are 3 parameters in this example • A parameter should start with the _____ of the variable, followed by the _____ • You want to avoid have a variable name that matches the name of the instance variables • Parameters are separated by a comma and can be in any order. You could have also written _____ • When we create an object, we will pass it some data values (a `String` first, an `int` second, and a `boolean` last) and that becomes the data for our object.	
End the Class	

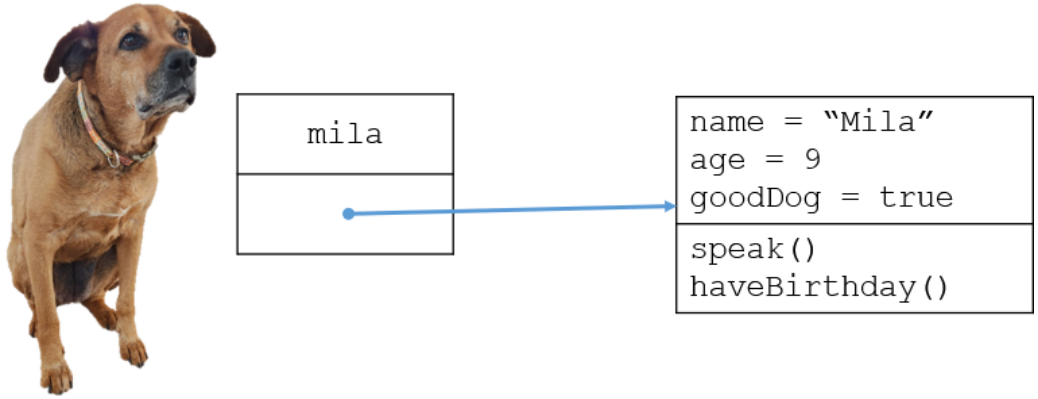
Here are some examples of other parameter constructors:

`public Dog(String n, int a)`
`{`
 	
`}`
	• This would be an example of a constructor you could use if you knew the name and age of the dog, but you didn't yet know if they were a `goodDog` or not yet.
	• Since you are not given some data for that variable, you want to set it equal to a default value.

`public Dog(boolean gd)`
`{`
 	
`}`
	• Here is another example of a parameter constructor.

Creating Objects

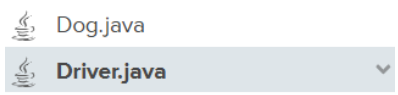
- All objects are stored using _____; when they are created, they point to a location in memory that contains the state (instance variables) as well as their values
- If an object is declared but not instantiated, it is `null`
- Once an object is created, the _____ is used when it needs to call on any method from the object creating class
- Creating objects takes place _____, usually in a `main` method of another class
- All objects are created with the _____ keyword, that is put right before the constructor call



Declaring an object	*Instantiating an object with the default constructor*	*Instantiating an object with the parameter constructor*
`Dog mila;`	`mila = new Dog();`	`Dog buddy = new Dog("Jax", 4, true);` `Dog ollie = new Dog("Olive", 10);`

Creating and Using a Class in the Same File

- When you write a custom class and the main method in the same file, Java automatically recognizes and allows the use of your class without the need for an import statement.
- This is because both the main method and your custom class are part of the same file and therefore belong to the same package by default.



Driver.java

```
1  public class Driver
2  {
3      public static void main(String[] args)
4      {
5          Dog mila;
6          mila = new Dog();
7          Dog buddy = new Dog("Jax", 4, true);
8          Dog ollie = new Dog("Olive", 10);
9      }
10 }
```

Dog.java

```
1  public class Dog
2  {
3      private String name;
4      private int age;
5      private boolean goodDog;
6
7      public Dog()
8      {
9          name = "None";
10         age = 0;
11         goodDog = true;
12     }
13     public Dog(String n, int a, boolean g)
14     {
15         name = n;
16         age = a;
17         goodDog = g;
18     }
19 }
```